International Journal of Systems Science
Vol. 40, No. 4, April 2009, 429–438

Taylor & Francis
Taylor & Francis Group

Optimisation analysis for reliability assessment based on stochastic differential equation modelling for open source software

Yoshinobu Tamura[a]* and Shigeru Yamada[b]

[a]*Faculty of Applied Information Science, Department of Computer Science, Hiroshima Institute of Technology, Hiroshima, Japan;* [b]*Department of Social Management Engineering, Graduate School of Engineering, Tottori University, Tottori, Japan*

(*Received 23 December 2007; final version received 16 September 2008*)

A new distributed software development paradigm typified by such open source project will evolve at a rapid pace in the future. Especially, open source software (OSS) which serve as key components of critical infrastructures in the social life are still ever-expanding now. We propose a software reliability growth model based on stochastic differential equations in order to consider the active state of the open source project. Especially, we assume that the software failure intensity depends on the time, and the software fault-reporting phenomena on the bug-tracking system keeps an irregular state. Moreover, it has been necessary to manage the software development process in terms of reliability, development effort and version-upgrade time. We find the optimal version-upgrade time based on the total expected software maintenance effort. We show that the proposed model can assist improvement of quality for OSS systems developed under the open source project.

Keywords: open source software; software reliability; stochastic differential equations; optimal total version-upgrade time

1. Introduction

Network technologies have made rapid progress with the dissemination of computer systems in all areas. These network technologies become increasingly more complex in a wide sphere. The mainstream of software development environment is the development paradigms such as concurrent distributed development environment and the so-called open source project by using network computing technologies. Especially, an open source software (OSS) system is frequently applied as server use, instead of client use. Such OSS systems which serve as key components of critical infrastructures in our social life are still ever-expanding now. The open source project contains special features like so-called software composition by which several geographically-dispersed components are developed in all parts of the world. The successful experience of adopting the distributed development model in such open source projects includes GNU/Linux operating system,[1] Apache Web server and so on[2] (E-soft Inc 2007). However, the poor handling of quality and customer support prohibit the progress of OSS. We focus on the problems in the software quality that prohibit the progress of OSS.

Especially, software reliability growth models (SRGMs) (Yamada 1994) have been applied to assess the reliability for quality management and testing progress control of software development. On the other hand, the effective method of dynamic testing management for new distributed development paradigms as typified by the open source project has only a few to be presented (Li, Shaw, Herbsleb, Ray, and Santhanam 2004; Zhoum, Davis 2005; Kuk 2006; MacCormack, Rusnak, and Baldwin 2006). In case of considering the effect of the debugging process on entire system in the development of a method of reliability assessment for OSS, it is necessary to grasp the situation of registration for bug-tracking system, the degree of maturation of OSS and so on.

In this article, we focus on an OSS developed under the open source project. We discuss a useful software reliability assessment method in open source project as a typical case of new distributed development paradigm. Especially, we propose a SRGM based on stochastic differential equations in order to consider the active state of the open source project. Especially, we assume that the software failure intensity depends on the time, and the software fault-reporting

*Corresponding author. Email: tam@cc.it-hiroshima.ac.jp

ISSN 0020–7721 print/ISSN 1464–5319 online
© 2009 Taylor & Francis
DOI: 10.1080/00207720802556245
http://www.informaworld.com

phenomena on the bug-tracking system keeps an irregular state. Moreover, we assume that the software failure intensity depends on the fault importance levels. Also, we analyse actual software fault-count data to show numerical examples of software reliability assessment for the OSS. We show that the proposed model can assist improvement of quality for OSS systems developed under the open source project. Furthermore, we derive the optimal total version-upgrade time based on the total expected software maintenance effort.

2. Stochastic differential equation modelling

Let $S(t)$ be the cumulative number of faults detected in the OSS project at operational time t ($t \geq 0$). Suppose that $S(t)$ takes on continuous real values. Since latent faults in the OSS system are detected and eliminated during the operational phase, $S(t)$ gradually increases as the operational procedures go on. Considering the characteristics of OSS development, the OSS developers report several related-faults when the OSS developers confirm the specific faults of bug-tracking system, i.e. the OSS developers can be the OSS users. Therefore, we assume that the increasing rate of $S(t)$ is proportional to the value $S(t)$ itself. Thus, under common assumptions for software reliability growth modelling, we consider the following linear differential equation:

$$\frac{\mathrm{d}S(t)}{\mathrm{d}t} = \lambda(t)S(t), \tag{1}$$

where $\lambda(t)$ is the intensity of inherent software failures at operational time t and a non-negative function.

In most cases, the detected faults of OSS are not reported to the bug-tracking system at the same time as fault detection but rather reported to the bug-tracking system with the time lag of fault detection and fault reporting. As for the fault reporting to the bug-tracking system, we consider that the software fault-reporting phenomena on the bug-tracking system keeps an irregular state. Moreover, the addition and deletion of software components are repeated under the development of OSS, i.e. we consider that the software failure intensity depends on the time.

Therefore, we suppose that $\lambda(t)$ in Equation (1) has the irregular fluctuation. That is, we extend Equation (1) to the following stochastic differential equation (Wong 1971; Arnold 1974):

$$\frac{\mathrm{d}S(t)}{\mathrm{d}t} = \{\lambda(t) + \sigma\gamma(t)\}S(t), \tag{2}$$

where σ is a positive constant representing a magnitude of the irregular fluctuation and $\gamma(t)$ a standardised Gaussian white noise.

We extend Equation (2) to the following stochastic differential equation of an Itô type (Wong 1971; Arnold 1974; Yamada, Kimura, Tanaka, and Osaki 1994):

$$\mathrm{d}S(t) = \left\{\lambda(t) + \frac{1}{2}\sigma^2\right\}S(t)\mathrm{d}t + \sigma S(t)\mathrm{d}W(t), \tag{3}$$

where $W(t)$ is a one-dimensional Wiener process which is formally defined as an integration of the white noise $\gamma(t)$ with respect to time t. The Wiener process is a Gaussian process and it has the following properties:

$$\Pr[W(0) = 0] = 1, \tag{4}$$

$$\mathrm{E}[W(t)] = 0, \tag{5}$$

$$\mathrm{E}[W(t)W(t')] = \mathrm{Min}[t, t']. \tag{6}$$

By using Itô's formula (Wong 1971; Arnold 1974), we can obtain the solution of Equation (2) under the initial condition $S(0) = v$ as follows (Yamada et al. 1994):

$$S(t) = v \cdot \exp\left(\int_0^t \lambda(s)\mathrm{d}s + \sigma W(t)\right), \tag{7}$$

where v is the total number of detected faults for the previous software version. Using solution process $S(t)$ in Equation (7), we can derive several software reliability measures.

Moreover, we define the intensity of inherent software failures, $\lambda(t)$, as follows:

$$\int_0^t \lambda_i(s)\mathrm{d}s = \sum_{i=1}^{n} p_i(1 - \exp[-\alpha_i t]), \tag{8}$$

where α_i ($i = 1, 2, \ldots, n$) is an acceleration parameter of the intensity of inherent software failures for the i-th fault importance level, p_i ($\sum_{i=1}^{n} p_i = 1$) the weight parameter for the i-th fault importance level, n the number of the applied fault importance levels, and $\lambda_i(s)$ is the software failure intensity for the i-th fault importance level at operational time s. We can apply the S-shaped growth curve function to Equation (8) depending on the trends of fault importance level.

3. Method of maximum likelihood

In this section, the estimation method of unknown parameters α and σ in Equation (7) is presented. Let us denote the joint probability distribution function of the process $S(t)$ as

$$P(t_1, y_1; t_2, y_2; \ldots; t_K, y_K) \equiv \Pr[S(t_1) \leq y_1, \ldots, S(t_K) \leq y_K | S(t_0) = v], \tag{9}$$

where $S(t)$ is the cumulative number of faults detected up to operational time t ($t \geq 0$). We denote the probability density as

$$p(t_1, y_1; t_2, y_2; \ldots; t_K, y_K) \equiv \frac{\partial^K P(t_1, y_1; t_2, y_2; \ldots; t_K, y_K)}{\partial y_1 \partial y_2 \ldots \partial y_K}. \quad (10)$$

Since $S(t)$ takes on continuous values, we construct the likelihood function l for the observed data (t_k, y_k) $(k = 1, 2, \ldots, K)$ as follows:

$$l = p(t_1, y_1; t_2, y_2; \ldots; t_K, y_K). \quad (11)$$

For convenience in mathematical manipulations, we use the following logarithmic likelihood function:

$$L = \log l. \quad (12)$$

The maximum-likelihood estimates α_i^* and σ^* are the values making L in Equation (12) to maximise. These can be obtained as the solutions of the following simultaneous likelihood equations (Yamada et al. 1994):

$$\frac{\partial L}{\partial \alpha_i} = \frac{\partial L}{\partial \sigma} = 0. \quad (13)$$

We can easily calculate Equations (12) and (13) by using the software such as Mathematica of Wolfram Research, Inc.

4. Software reliability assessment measures

4.1. *Expected number of detected faults and their variances*

We consider the mean number of faults detected up to operational time t. The density function of $W(t)$ is given by

$$f(W(t)) = \frac{1}{\sqrt{2\pi t}} \exp\left\{-\frac{W(t)^2}{2t}\right\}. \quad (14)$$

Information on the current number of detected faults in the system is important to estimate the situation of the progress on the operational procedures. Since it is a random variable in our model, its expected value and variance can be useful measures. We can calculate them from Equation (7) as follows (Yamada et al. 1994):

$$E[S(t)] = v \cdot \exp\left(\int_0^t \lambda(s)\mathrm{d}s + \frac{\sigma^2}{2} t\right), \quad (15)$$

$$\mathrm{Var}[S(t)] = E[\{S(t) - E[S(t)]\}^2] = v^2 \cdot \exp\left(2\int_0^t \lambda(s)\mathrm{d}s + \sigma^2 t\right)\{\exp(\sigma^2 t) - 1\}, \quad (16)$$

where $E[S(t)]$ is the expected number of faults detected up to time t.

From Equation (15), we can confirm that the expected number of detected faults cannot converge to a finite value as the following equation:

$$\lim_{t \to \infty} E[S(t)] = \infty. \quad (17)$$

The operation environment of OSS has the characteristics of the susceptible to various operational environments. Therefore, it is different from conventional software systems developed under the identical organisation. Then, the expected number of detected faults continues to increase from the effect of the interaction among various operational environments, i.e. the expected number of detected faults cannot converge to a finite value (Tamura and Yamada 2005a, b, 2006a, b, 2007).

4.2. *Mean times between software failures*

The instantaneous mean time between software failures (denoted by $\mathrm{MTBF_I}$) is useful to measure the property of the frequency of software failure-occurrence.

Instantaneous MTBF is approximately given by

$$\mathrm{MTBF_I}(t) = \frac{1}{E\left[\frac{\mathrm{d}S(t)}{\mathrm{d}t}\right]}. \quad (18)$$

Therefore, we have the following instantaneous MTBF:

$$\mathrm{MTBF_I}(t) = \frac{1}{v\left(\lambda(t) + \frac{1}{2}\sigma^2\right) \cdot \exp\left(\int_0^t \lambda(s)\mathrm{d}s + \frac{\sigma^2}{2} t\right)}. \quad (19)$$

Also, the cumulative mean time between software failures (denoted by $\mathrm{MTBF_C}$) is approximately given by

$$\mathrm{MTBF_C}(t) = \frac{t}{E[S(t)]}. \quad (20)$$

Therefore, we have the following cumulative MTBF:

$$\mathrm{MTBF}_C(t) = \frac{t}{v \cdot \exp\left(\int_0^t \lambda(s)\mathrm{d}s + \frac{\sigma^2}{2} t\right)}. \quad (21)$$

4.3. *Coefficient of variation*

Also, we can derive the following coefficient of variation from Equation (7):

$$\mathrm{CV}(t) \equiv \frac{\sqrt{\mathrm{Var}[S(t)]}}{E[S(t)]}, \quad (22)$$

Table 1. Schedule of release candidate version in Fedora Core 7.

Date	Event
1 February 2007	Test 1 Release
29 February 2007	Test 2 Release
27 March 2007	Test 3 Release
24 April 2007	Test 4 Release
31 May 2007	Fedora 7 General Availability

which shows the stability of debugging process in the OSS project.

5. Numerical illustrations

5.1. *Data for numerical illustrations*

We focus on the Fedora Core Linux (Red Hat Inc. 2007) which is one of the operating systems developed under an open source project. The Fedora project is made up of many small-size projects. Fedora is a set of projects, sponsored by Red Hat and guided by the Fedora Project Board.[3] These projects are developed by a large community of people who strive to provide and maintain the very best in free, OSS and standards.

We can obtain the fault-count data from the Red Hat bug-tracking system (Red Hat Inc. 2008). The fault-count data used in this article are collected in the bug-tracking system on the website in May 2007. Especially, we focus on the Kernel component of the Fedora Core Linux. The schedule of release candidate versions in Fedora Core 7 is shown in Table 1.

As shown in Table 1, we focus on the release candidate version. The faults reported during release candidate version can be considered synonymous with the faults detected during the testing phase of the conventional software system developed under the identical organisation. We estimate the growth curve by using the data from Test 1 to Test 3 in Table 1. Then, we can estimate the optimal version-upgrade time for final release as shown in Section 6.

5.2. *Reliability assessment results considering fault level*

Figure 1 is shown in the number of detected faults for each fault importance level. The number of detected faults for the end of the Test 1 release version is 48 faults. The number of detected faults for 'High' level at the end of fault-reporting on Test 3 is 48 faults, the one of 'Medium' level is 105 faults. Therefore, we analyse the actual data about the case of $v = 69$, $p_1 = 48/153 = 0.31373$, $p_2 = 1 - p_1 = 0.68627$, where v is the number of detected faults before the release

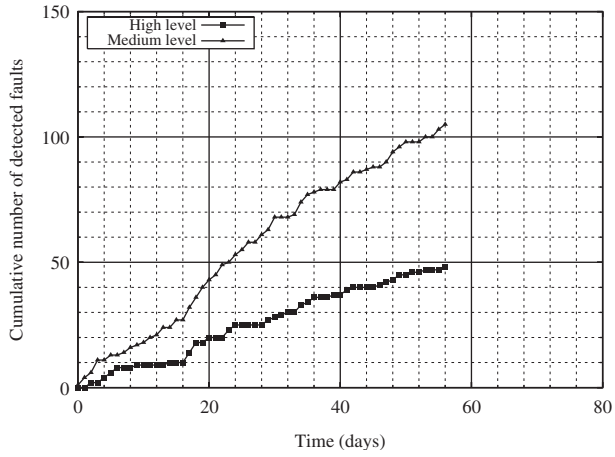


Figure 1. The number of detected faults for each fault level.

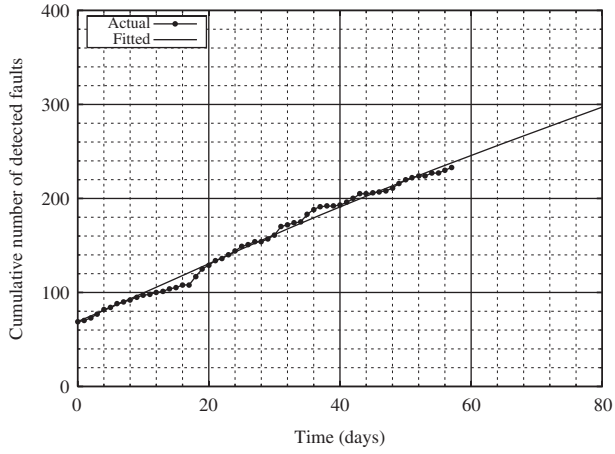


Figure 2. The estimated number of detected faults, $\hat{E}[S(t)]$.

candidate version (Test 1 Release), p_1 the weight of high level, and p_2 the weight of medium level.

The following model parameters have been estimated by solving the likelihood equations given in Equation (13):

$$\hat{\alpha_1} = 0.035625,\ \hat{\alpha_2} = 0.035509,\ \hat{\sigma} = 0.11385.$$

The estimated expected number of detected faults in Equation (15), $\hat{E}[S(t)]$, is shown in Figure 2.

Also, Figures 3 and 4 show the estimated software failure intensity, Equation (8) and $\lambda(t) = p_i\alpha_i \exp[-\alpha_i t]$, respectively. From Figures 3 and 4, we can confirm the changes characteristic of the estimated software failure intensity for each fault importance level.

Figure 5 shows the estimated variance of the number of faults in Equation (16), $\widehat{\mathrm{Var}}[S(t)]$. In Figure 5, it is shown that the variance of the number of detected faults grows as the time elapses after the evaluation version of Fedora Core 7 has been released.

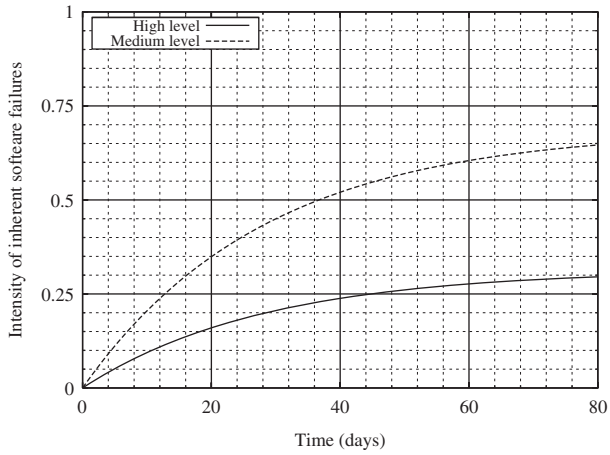


Figure 3. The estimated intensity of inherent software failures for each fault level, $\{p_i(1-\exp[-\alpha_i t])\}$.

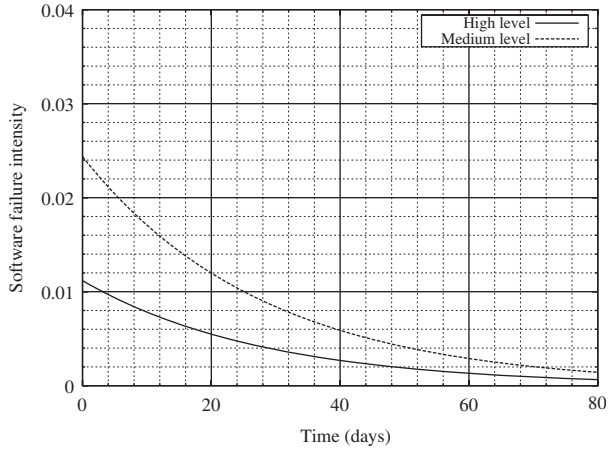


Figure 4. The estimated software failure intensity for each fault level, $(p_i\alpha_i \exp[-\alpha_i t])$.

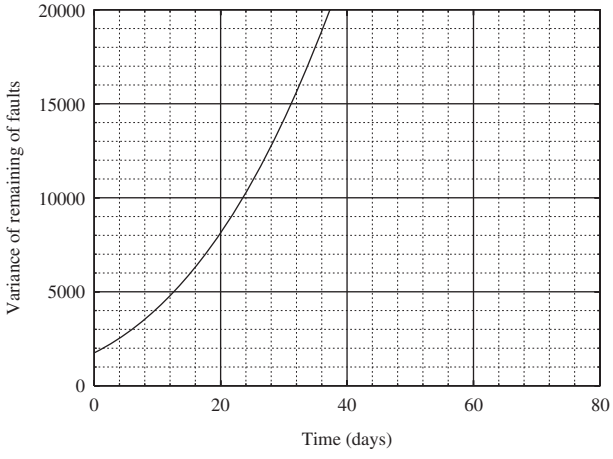


Figure 5. The estimated variance of the number of detected faults, $\widehat{\mathrm{Var}}[S(t)]$.

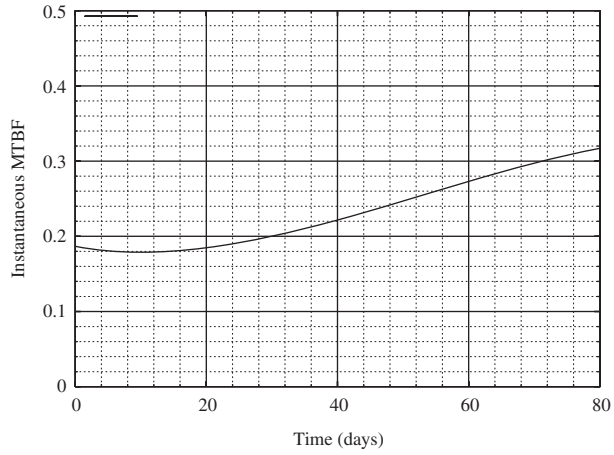


Figure 6. The estimated $\mathrm{MTBF_I}$, $\widehat{\mathrm{MTBF}}_I(t)$.

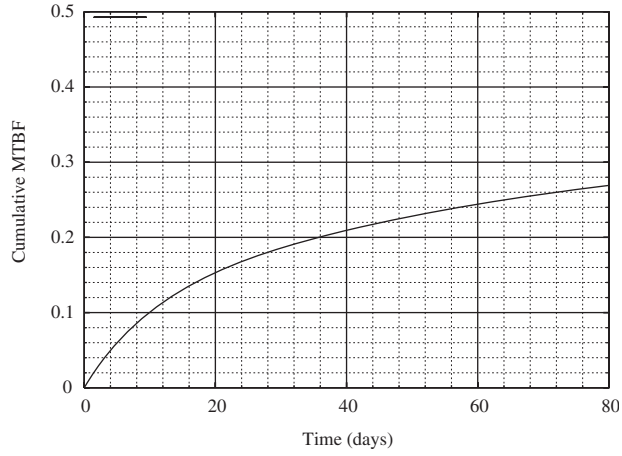


Figure 7. The estimated $\mathrm{MTBF_C}$, $\widehat{\mathrm{MTBF}}_C(t)$.

Moreover, the estimated $\mathrm{MTBF_I}$ in Equation (19), $\widehat{\mathrm{MTBF}}_I(t)$, and the estimated $\mathrm{MTBF_C}$ in Equation (21), $\widehat{\mathrm{MTBF}}_C(t)$, are also plotted in Figures 6 and 7, respectively. These figures show that the MTBF increase as the operational procedures go on.

5.3. *Sensitivity analysis in terms of model parameters*

From the results of the former sections, we have verified that our model can be applied to quantitatively assess software reliability in the operational phase of the OSS. In this section, we show several behaviour of software reliability assessment measures if we change parameters σ and α which are the magnitude of the irregular fluctuation and the acceleration parameter of the intensity of initial inherent failure.

In addition to the case of $(\alpha_1+\alpha_2)/2=0.035567$ and $\sigma=0.11385$ in the former section, we represent the estimated mean value function with changing the

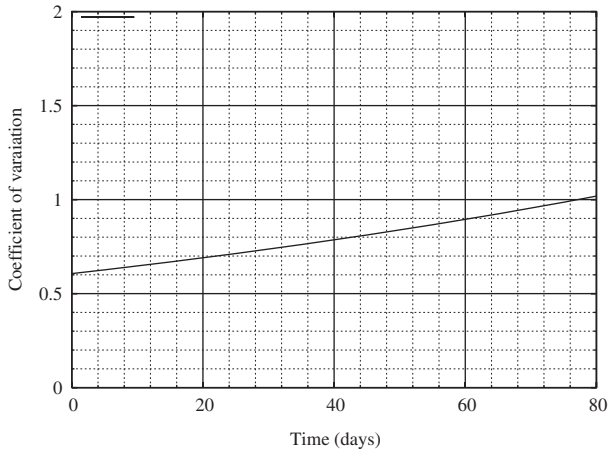


Figure 8. The estimated coefficient of variation, CV(t).

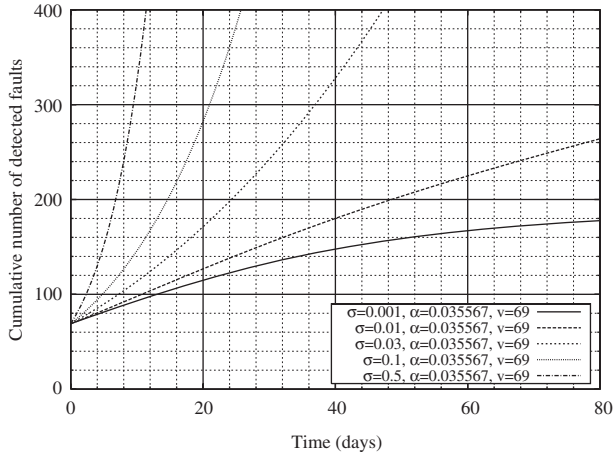


Figure 10. Dependence of model parameter σ.

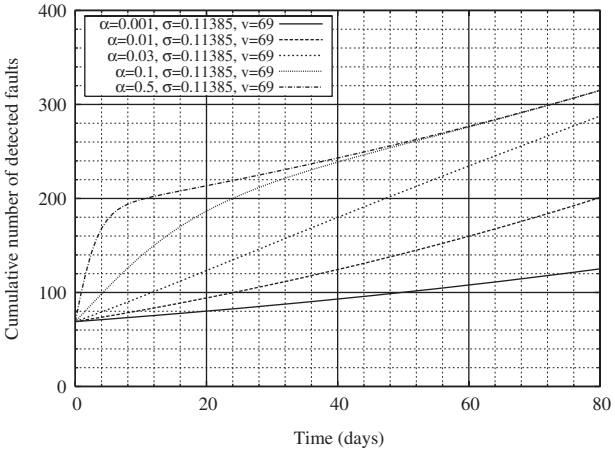


Figure 9. Dependence of model parameter α.

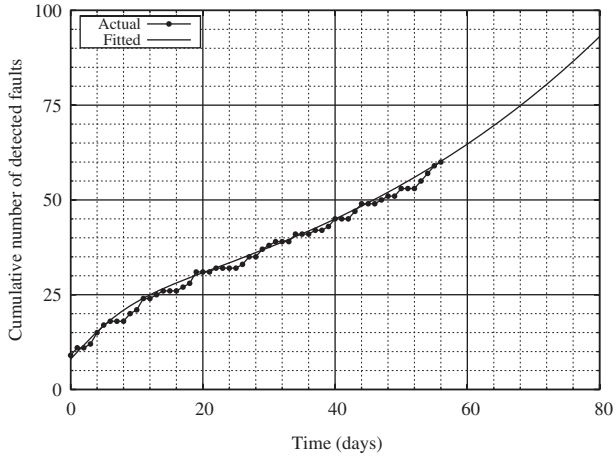


Figure 11. The estimated number of detected faults for Apache HTTP server, $\hat{E}[S(t)]$.

values of parameters σ and α at regular intervals in Figures 9 and 10, respectively.

5.4. *Performance evaluation of our model*

There are many open source projects around the world. Especially, we focus on operating system, server software and application software. We have shown several reliability assessment results for operating system software in the former section. Therefore, we show the reliability assessment results for the OSS's of server and application in order to evaluate our model performance.

We show the numerical example of Apache HTTP server.[4] Apache HTTP server has the largest market share in the server software (E-Soft Inc. 2007). Moreover, we show the numerical examples of the Firefox Web browser developed under the Mozilla.org project.

The estimated number of detected faults for version 2.2.3 of Apache HTTP server is shown in Figure 11. Moreover, we show the estimated number of detected faults for version RC3 Firefox 2 in Figure 12.

In the above-mentioned results, we can confirm that the reliability growth curves of the server software and application software exponentially grow with time elapses. Also, we found that our model can cover an increase in the number of detected faults soon after the version upgrade from Figure 11.

5.5. *Goodness-of-fit comparison*

We compare the goodness-of-fit of our model discussed in this article. We adopt the value of mean square error (MSE) as comparison criteria of goodness-of-fit (Iannino, Musa, Okumoto, and Littlewood 1984). We consider that the comparison criteria such as the

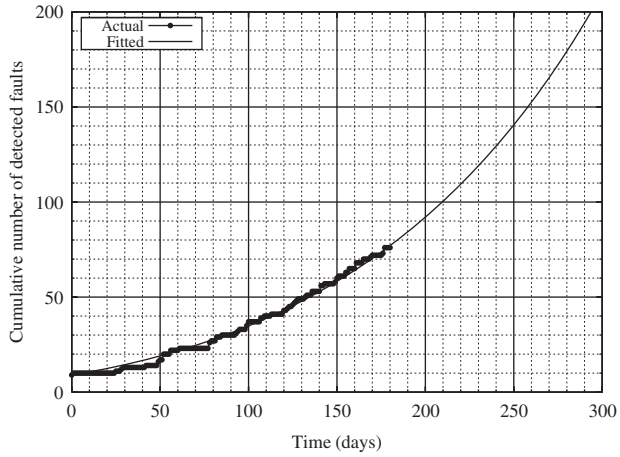


Figure 12. The estimated number of detected faults for Firefox, $\hat{E}[S(t)]$.

Table 2. Results of goodness-of-fit comparison.

Compared models	Fedora Core 6	Apache HTTP server	Firefox
Our SDE model	19.986	1.5447	3.7026
Exponential SDE model	22.078	4.1234	57.280
LPET model	22.515	3.9030	9.8863
WP model	30.525	56.931	–

Akaike's Information Criterion (AIC) is an improper one, because of two different types of stochastic process, i.e. NHPP and SDE. We compare our stochastic differential equation model (Our SDE model) with the existing exponential stochastic differential equation model (Exponential SDE model) (Yamada et al. 1994), logarithmic Poisson execution time model (LPET model) (Musa and Okumoto 1984), Weibull process model (WP model) (Yamada 1994).

We have applied the above-mentioned NHPP models because the cumulative number of detected faults tend to not converge in the case of OSS systems (Li, Shaw, Herbsleb, Ray, and Santhanam 2004; Zhou and Davis 2005). Therefore, it is difficult to estimate the growth curve of OSS by using NHPP models assumed that the detectable fault number is finite value.

First, Table 2 shows the result of goodness-of-fit comparison in terms of the MSE for the observed data. We found that our model fits best among the compared models with respect to the MSE.

6. Optimal version-upgrade problem

Recently, it becomes more difficult for software developers to produce highly reliable software systems efficiently, because of the more diversified and complicated software requirements. Thus, it has been necessary to control the software development process in terms of reliability, cost and delivery time. On the other hand, the effective optimal software version-upgrade problem for OSS has only a few presented. It is very important in terms of software management that we decide for the optimal length of testing versions for OSS. We find the optimal testing-time based on the total expected software maintenance effort in this section. Several optimal software release problems considering host-concentrated software development process have been proposed by many researchers (Yamada, Narihisa, and Osaki 1984; Yamada and Osaki 1987; Pham 1996; Chatterjee, Misra, and Alam 1997). However, optimal software release problems for OSS have not been proposed. Therefore, we formulate a maintenance effort model based on our SRGM discussed in Section 2, and analyse the optimal release problem minimising the total expected maintenance effort.

It is interesting for the software developers to predict and estimate the time when we should stop testing in order to develop a highly reliable software system efficiently. Hence, we discuss the method of determination of software version-upgrade times minimising the total expected development effort.

We define the following:

m_0: the fixing effort per fault during the test-version,
m_1: the maintenance effort per fault during the test-version,
m_2: the effort per time for fixing faults during the test-version.

Then, the expected software effort in the test-version of OSS can be formulated as

$$E_1(t) = m_0 E[S(t)]. \tag{23}$$

Also, the expected software maintenance effort after the release of general availability is given as follows:

$$E_2(t) = m_1\{E[S(t_p)] - E[S(t)]\} + m_2 t, \tag{24}$$

where t_p is the previous version-upgrade period.

Moreover, if the software components are added to entire system after the version upgrade, the penalty effort is imposed. We define the penalty effort function as follows:

$$G(t) = (1-c)\exp\left[\frac{t-t_p}{v}\right], \tag{25}$$

where c is the ratio of the number of new components to entire system after version upgrade, and v the number of prior version upgrade.

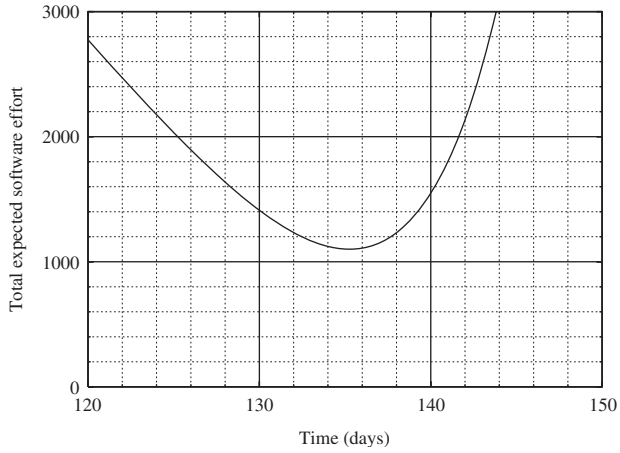


Figure 13. The estimated total expected software effort.

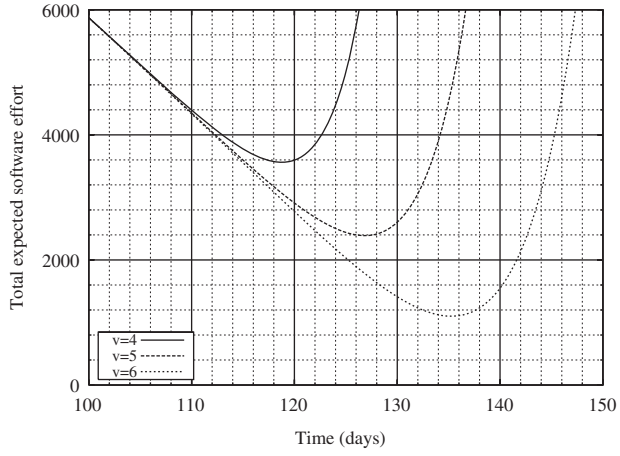


Figure 14. Dependence of model parameter v for the total expected software effort.

Consequently, from Equations (23), (24) and (25), the total expected software effort is given by

$$E(t) = E_1(t) + E_2(t) + G(t). \tag{26}$$

The optimum version-upgrade time t^* can be obtained by minimising $E(t)$.

6.1. *Numerical illustrations for optimal version-upgrade time*

In this section, we show several numerical examples based on the optimal version-upgrade problems. Figure 13 is shown the estimated total expected software effort where $t_p = 92$, $v = 6$, $c = 0.17$, $m_0 = 76$, $m_1 = 87$ and $m_2 = 163$. t_p is the previous version-upgrade period from Test 1 to General Availability of Fedora Core 6. Also, v is the number of historical version-upgrade, i.e. the prior version means

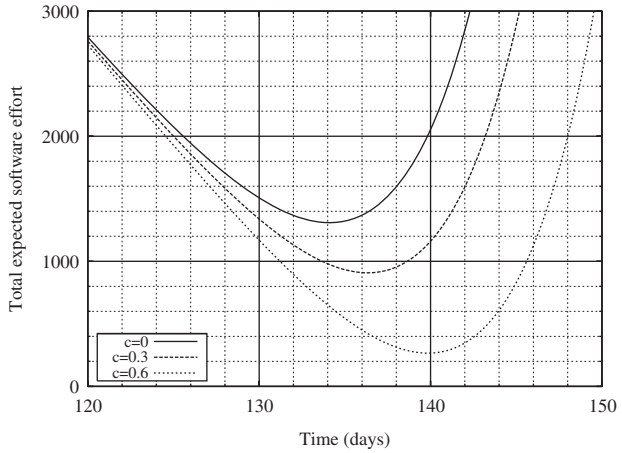


Figure 15. Dependence of model parameter c for the total expected software effort.

Fedora Core 6. $c = 1/6 = 0.17$ means that new Kernel-xen-2.6 component is added as sixth component. The unit of m_0, m_1 and m_2 is the man-days effort. From Figure 13, we find that the optimum version-upgrade time is derived as $t^* = 135$ days from Figure 13. Then, the total expected software effort $\hat{E}(t^*)$ is 1122.6 man–days effort.

Moreover, we have verified that our effort model can be applied to estimate version-upgrade time of the OSS. We show some behaviours of our effort model if we change the parameters v and c which are the number of prior version-upgrade and the ratio of the number of new components to entire system after version upgrade. In addition to the case of $v = 6$ and $c = 0$, we represent the estimated total expected software effort with changing the value of the parameter v and c at regular intervals are illustrated in Figures 14 and 15, respectively. From Figures 14 and 15, we have found that the total expected software effort decreases with the growth of parameters v and c. On the other hand, the optimal version-upgrade time increases with the growth of parameters v and c.

7. Concluding remarks

In this article, we have focussed on the Fedora Core Linux operating system which is known as the OSS, and discussed the method of reliability assessment for the OSS developed under on an open source project. Moreover, we have proposed a SRGM based on stochastic differential equations in order to consider the active state of the open source project. Especially, we have assumed that the software failure intensity depends on the time, and the software fault-reporting phenomena on the bug-tracking system keep an irregular state. Also, we have analysed actual software

fault-count data to show numerical examples of software reliability assessment for the OSS.

Furthermore, it has been necessary to control the software development process in terms of reliability, development effort and version-upgrade time for OSS. We have formulated the maintenance effort model based on our SRGM and analysed the optimal release problem minimising the total expected maintenance effort. Also, we have derived the optimum version-upgrade time.

Finally, we have focussed on an OSS developed under open source projects. New distributed development paradigm typified by such open source project will evolve at a rapid pace in the future. Our method is useful as the method of reliability assessment for an entire system of OSS.

In future, we will develop the software reliability assessment tool (Tamura, Hadatsuki, Yamada, and Kimura 2006) based on our model.

Acknowledgements

This work was supported in part by the Grant-in-Aid for Scientific Research (C), Grant No. 18510124 and Young Scientists (B), Grant No. 17700039 from the Ministry of Education, Culture, Sports, Science and Technology of Japan, and the Grant-in-Aid for Research from Satake Foundation for Technology.

Notes

1. Linux is a registered trademark of Linus Torvalds.
2. Other company, product or service names may be trademarks or service marks of others.
3. Fedora is a trademark of Red Hat, Inc. The Fedora Project is not a supported product of Red Hat, Inc.
4. The Apache HTTP server is developed under the Apache Software Foundation.

References

Arnold, L. (1974), *Stochastic Differential Equations – Theory and Applications*, New York: John Wiley & Sons.

Chatterjee, S, Misra, R.B., and Alam, S.S. (1997), 'Joint Effect of Test Effort and Learning Factor on Software Reliability and Optimal Release Policy,' *International Journal of System Science*, 28, 391–396.

E-Soft Inc. (2007), 'Internet Research Reports,' Available from: http://www.securityspace.com/s_survey/data/index.html

Iannino, A., Musa, J.D., Okumoto, K., and Littlewood, B. (1984), 'Criteria for Software Reliability Model Comparisons,' *IEEE Transactions on Software Engineering*, SE-10, 687–691.

Kuk, G. (2006), 'Strategic Interaction and Knowledge Sharing in the KDE Developer Mailing List,' *INFORMS Journal of Management Science*, 52, 1031–1042.

Lawrie, T., and Gacek, C. (2002), 'Issues of Dependability in Open Source Software Development,' *SIGSOFT Software Engineering Notes*, 27, 34–37.

Lee, C., Kim, Y., and Park, D. (2004), 'S-shaped Software Reliability Growth Models Derived from Stochastic Differential Equations,' *IIE Transactions*, 36(12), 1193–1199.

Li, P., Shaw, M., Herbsleb, J., Ray, B., and Santhanam, P. (2004), 'Empirical Evaluation of Defect Projection Models for Widely-deployed Production Software Systems,' in *Proceedings of the 12th International Symposium on the Foundations of Software Engineering (FSE-12)*, pp. 263–272.

MacCormack, A., Rusnak, J., and Baldwin, C.Y. (2006), 'Exploring the Structure of Complex Software Designs: An Empirical Study of Open Source and Proprietary Code,' *INFORMS Journal of Management Science*, 52, 1015–1030.

Musa, J.D., and Okumoto, K. (1984), 'A Logarithmic Poisson Execution Time Model for Software Reliability Measurement,' in *Proceedings of the 7th International Conference on Software Engineering*, pp. 230–238.

Pham, H. (1996), 'A Software Cost Model With Imperfect Debugging Random Life Cycle and Penalty Cost,' *International Journal System Science*, 27, 455–463.

Red Hat, Inc. and others (2007), 'Fedora Project,' Available from: http://fedora.redhat.com/

Red Hat, Inc. (2007), 'Bugzilla Main Page,' Available from: http://bugzilla.redhat.com/

Tamura, Y., Hadatsuki, K., Yamada, S., and Kimura, M. (2006), 'Development of a User-oriented Reliability Assessment Tool for Open Source Software (In Japanese),' in *Proceedings of the Linux Conference 2006*, p. 4. Available from: http://lc.linux.or.jp/lc2006/

Tamura, Y., and Yamada, S. (2005a), 'Comparison of Software Reliability Assessment Methods for Open Source Software,' in *Proceedings of the 11th IEEE International Conference on Parallel and Distributed Systems (ICPADS2005) – Volume II*, pp. 488–492.

Tamura, Y., and Yamada, S. (2005b), 'Validation of an OSS Reliability Assessment Method Based on ANP and SRGM's,' *Proceedings of the International Workshop on Recent Advances in Stochastic Operations Research*, 273–280.

Tamura, Y., and Yamada, S. (2006a), 'A Method of User-oriented Reliability Assessment for Open Source Software and Its Applications,' in *Proceedings of the 2006 IEEE International Conference on Systems, Man, and Cybernetics*, pp. 2185–2190.

Tamura, Y., and Yamada, S. (2006b), 'A Method of User-oriented Reliability Assessment for Open Source Software and Its Applications,' in *Proceedings of the 2006 IEEE International Conference on Systems, Man, and Cybernetics*, pp. 2185–2190.

Tamura, Y., and Yamada, S. (2007), 'Software Reliability Assessment and Optimal Version-Upgrade Problem for Open Source Software,' in *Proceedings of the 2007 IEEE International Conference on Systems, Man, and Cybernetics*, pp. 1333–1338.

Wong, E. (1971), *Stochastic Processes in Information and Systems*, New York: McGraw-Hill.

Yamada, S. (1994), *Software Reliability Models: Fundamentals and Applications (in Japanese)*, Tokyo: JUSE Press.

Yamada, S., Kimura, M., Tanaka, H., and Osaki, S. (1994), 'Software Reliability Measurement and Assessment with Stochastic Differential Equations,' *IEICE Transactions on Fundamentals*, E77-A, 109–116.

Yamada, S., Narihisa, H., and Osaki, S. (1984), 'Optimal Release Policies for a SOftware System with a SCheduled Software Delivery Time,' *International Journal of Systems Science*, 15, 905–914.

Yamada, S., and Osaki, S. (1987), 'Optimal Software Release Policies with Simultaneous Cost and Reliability Requirements,' *European Journal of Operational Research*, 31, 46–51.

Zhou, Y., and Davis, J. (2005), 'Open Source Software Reliability Model: An Empirical Approach,' in *Proceedings of the Fifth Workshop on Open Source Software Engineering*, New York: ACM, pp. 1–6.

Zhoum, Y., and Davis, J. (2005), 'Open Source Software Reliability Model: An Empirical Approach,' in *Proceedings of the Workshop on Open Source Software Engineering (WOSSE)*, 30(4), 67–72.